

گزارش فرآیند ایجاد پردازنده‌ها در سیستم‌عامل xv6

مقدمه

فرآیند ایجاد پردازنده (Process Creation) یکی از اصلی‌ترین وظایف سیستم‌عامل است. در سیستم‌عامل آموزشی xv6، این کار از طریق دو مکانیزم زیر انجام می‌شود:

۱. ایجاد نخستین پردازنده کاربر با استفاده از تابع `userinit`

۲. ایجاد پردازنده‌های جدید با فراخوانی سیستمی `fork`

در این گزارش، این دو مکانیزم را به همراه بررسی کدهای منبع مربوطه تحلیل می‌کنیم.

۱. ایجاد نخستین پردازنده:

مراحل اجرای `userinit`:

- تخصیص پردازنده:

ابتدا با استفاده از تابع `allocproc()`، یک ساختار پردازنده (نوع `struct proc`) ایجاد می‌شود. این ساختار شامل `trapframe` و یک جدول صفحه جدید است.

- بارگذاری کد اولیه:

محتوای `initcode` در فضای آدرس پردازش بارگذاری می‌شود. این کد برنامه‌
`init/` را با استفاده از `exec` اجرا می‌کند.

تنظیم مقادیرهای اولیه:

`p->trapframe->epc = 0` // نقطه شروع اجرای کد
`p->trapframe->sp = PGSIZE` // مقدار اولیه stack pointer

- تنظیمات دیگر:

نام پردازش روی `"initcode"` تنظیم می‌شود و وضعیت آن به `RUNNABLE` تغییر می‌یابد.

این تابع فقط یک بار و هنگام راه‌اندازی سیستم اجرا می‌شود.

۲. ایجاد پردازش جدید با فراخوانی `fork`

مراحل اجرای `fork`:

- تخصیص پردازه فرزند:

مانند `userinit`، از تابع `allocproc()` برای ایجاد یک ساختار پردازه جدید استفاده می‌شود.

- کپی حافظه کاربر:

تابع `uvmcopy` فضای حافظه کاربر والد را به فرزند کپی می‌کند.

- کپی وضعیت اجرای والد:

محتویات `trapframe` پردازه والد به فرزند منتقل می‌شود و مقدار رجیستر `a0` در فرزند برابر با `0` تنظیم می‌گردد (مطابق با قرارداد بازگشتی `fork`).

- تنظیمات پایانی:

یک شناسه پردازه (`pid`) منحصر به فرزند اختصاص داده می‌شود و وضعیت آن به `RUNNABLE` تغییر می‌کند.

گزارش زمان‌بندی پردازه‌ها

۱. ساختارهای داده پایه

الف) ساختار پردازش (ساختار **proc**)

- محل تعریف: در فایل‌های **proc.h** و **proc.c**

- وضعیت‌های ممکن برای پردازش:

```
} enum procstate
```

```
UNUSED, // پردازش تخصیص نیافته
```

```
USED, // در حال ایجاد
```

```
SLEEPING, // در انتظار رویداد
```

```
RUNNABLE, // آماده اجرا
```

```
RUNNING, // در حال اجرا روی CPU
```

```
ZOMBIE // خاتمه یافته
```

```
};
```

○ **kstack**: پشته کرنل پردازش

◦ `trapframe` : وضعیت ثبات‌های کاربر در هنگام وقفه

◦ `context` : زمینه اجرا هنگام سوئیچ پردازها

ب) آرایه پردازها

- آرایه گلوبال : `[proc[NPROC]`
- استفاده از قفل هم‌زمانی: `ptable.lock` برای محافظت در برابر دسترسی هم‌زمان توسط چند CPU

۲. الگوریتم زمان‌بندی (Round-Robin)

محل پیاده‌سازی:

تابع `scheduler()`

منطق اجرا:

در الگوریتم Round-Robin، هسته سیستم‌عامل به صورت چرخشی به پردازه‌هایی که در وضعیت `RUNNABLE` هستند زمان CPU اختصاص می‌دهد. این الگوریتم ساده و غیر اولویت‌دار است.

مراحل اجرا:

1. پیمایش آرایه `proc[]` برای یافتن پردازهای با وضعیت `RUNNABLE`

2. تغییر وضعیت پردازش به `RUNNING`

3. سوئیچ زمینه اجرا (Context Switch) با استفاده از دستور زیر:

```
;(swtch(&c->context, &p->context
```

4. پس از اتمام اجرای پردازش، بازگشت به زمان‌بند

حالت Idle (بی‌کاری):

```
asm volatile("wfi"); // Wait For Interrupt
```

این دستور باعث می‌شود CPU منتظر وقفه بماند تا دوباره فعالیت خود را آغاز کند.

۳. تعامل با وقفه‌ها و Preemption

پیش‌گیرندگی (Preemption):

- با استفاده از وقفه تایمر که تقریباً هر ۱۰۰ میلی‌ثانیه فعال می‌شود، سیستم‌عامل می‌تواند کنترل CPU را از پردازش جاری بازپس گیرد.

- این رفتار توسط تابع `yield()` مدیریت می‌شود

دیاگرام تغییر وضعیت پردازش‌ها:

()UNUSED --> RUNNABLE: allocproc

RUNNABLE --> RUNNING: scheduler انتخاب کند

RUNNING --> RUNNABLE: yield() / preemption

()RUNNING --> SLEEPING: sleep

()SLEEPING --> RUNNABLE: wakeup

()RUNNING --> ZOMBIE: exit

()ZOMBIE --> UNUSED: wait

مکانیزم Context Switch در xv6

مفهوم پایه

سوئیچ کانتکست فرآیند حیاتی برای:

- جابجایی بین پردازش‌های کاربر
- انتقال کنترل به زمان‌بند (scheduler)
- پشتیبانی از چندوظیفگی پیشگیرانه

اجزای کلیدی

1. ساختار `context`

- ذخیره حداقل حالت اجرا (رجیسترهای callee-saved)

- شامل:

```
} struct context
```

```
uint64 ra // آدرس بازگشت
```

uint64 sp // اشاره‌گر پشته

uint64 s0-s11 // رجیسترهای ذخیره شده

;

2. تابع `switch` (اسمبلی):

- ذخیره/بازیابی رجیسترهای حیاتی

- تغییر پشته کرنل (kernel stack)

فرآیند اجرا

1. ذخیره حالت فعلی:

- غیرفعال کردن وقفه‌ها (`intr\_off`)

- ذخیره `sp`, `ra` و رجیسترهای s0-s11

2. بارگذاری حالت جدید:

- بازیابی رجیسترهای پردازنده مقصد

- پرش به آدرس بازگشت ذخیره شده (`ra`)

3. مدیریت وقفه‌ها:

- بازگردانی وضعیت وقفه (`intena`)

- فعال کردن وقفه‌ها پس از سوئیچ

گزارش فرآیند خاتمه پردازش در xv6

مقدمه

فرآیند خاتمه پردازش‌ها در xv6 از طریق فراخوانی سیستمی `exit` و `wait` مدیریت می‌شود. این مکانیزم تضمین می‌کند منابع سیستم به درستی آزاد شده و از ایجاد پردازش‌های یتیم جلوگیری شود.

مراحل کلیدی خاتمه پردازش:

1. فراخوانی `exit`

- بستن تمام فایل‌های باز

- آزادسازی حافظه کاربر

- انتقال فرزندان به پردازه ``init`` (تابع ``reparent``)

- تغییر حالت به ``ZOMBIE`` و انتظار برای والد

2. فراخوانی ``wait`` - بازیابی وضعیت خروج فرزند

- آزادسازی نهایی منابع (تابع ``freeproc``)

گزارش Trapframe در سیستم عامل xv6

۱. نقش Trapframe

Trapframe تصویر کاملی از وضعیت CPU در لحظه وقوع یک `trap/interrupt/syscall` را نگه می‌دارد تا کرنل پس از سرویس‌دهی بتواند دقیقاً به همان نقطه در فضای کاربر بازگردد. این سازوکار، اجرای بی‌وقفه پردازها را—`even` با وقفه‌ها و خطاهای مکرر—تضمین می‌کند.

۲. زمان ایجاد

تخصیص هر پردازش هنگام ایجاد در `allocproc` یک

Trapframe دریافت می‌کند.

نگاشت Trapframe در `proc_pagetable` به فضای

آدرس پردازش نگاشت می‌شود.

اپدیت در هر وقفه/سیستم‌کال، کرنل مقادیر جدید را در

Trapframe می‌نویسد.

بازگشت به تابع `usertrapret` از Trapframe برای بازیابی

کاربر رجیسترها و اجرای دستور `sret/iret` استفاده

می‌کند.

۳. اجزای کلیدی Trapframe

دسته	مهم‌ترین فیلدها	هدف
رجیسترهای عمومی	(RISC-V <code>a0-a7, t0-t6</code>) یا (x86 <code>EAX, EBX, ...</code>)	نگهداری داده‌ها و نتیجه <code>system call</code>
کنترلی	<code>epc / EIP, EFLAGS, CS</code>	آدرس دستور بعدی و فلگ‌های وضعیت پردازنده

پشته کاربر	<code>sp / ESP, SS</code>	ادامه اجرای استک کاربر پس از بازگشت
کرل	<code>kernel_sp,</code> <code>kernel_hartid</code>	اطلاعات لازم برای برگشت به کرل در trap بعدی
شماره Trap و	<code>trapno, err</code>	تشخیص نوع وقفه یا خطای خطا
		صفحه

ترتیب قرارگیری فیلدها دقیقاً همان ترتیبی است که سخت افزار یا کد اسمبلی (`trapasm.S`) آن‌ها را روی پشته کرل می نویسد؛ این تطابق، بازیابی بی نقص را ممکن می کند.

۴. جریان کار خلاصه

1. وقوع Trap → سخت افزار بخشی از رجیسترها را روی پشته کرل می ریزد.
2. کد اسمبلی باقیمانده رجیسترها را ذخیره کرده و به C وارد می شود.
3. کرل سرویس لازم را انجام می دهد (`scheduler`، `syscall`، ...).
4. `usertrapret` مقادیر Trapframe را در رجیسترها برمی گرداند.

5. دستور `sret/iret` اجرا می‌شود و CPU دقیقاً در دستور `epc/EIP` کاربر از سر می‌گیرد.
